Parallel Computing, 2026S

OpenMP - Part I

Dr. João Ferreira Alves

Research Group Scientific Computing, University of Vienna

Literature, and Acknowledgments

OpenMP specification, tutorials, articles, etc. see **https://www.openmp.org**

Using OpenMP - Portable Shared Memory Parallel Programming. B. Chapman and G. Jost, R. van der Pas. MIT Press, 2007.

Using OpenMP—The Next Step. Affinity, Accelerators, Tasking, and SIMD. R. van der Pas, E. Stotzer, and C. Terboven. MIT Press, 2017.

The OpenMP Common Core. Timothy G. Mattson, Yun (Helen) He, and Alice E. Koniges

This material is partially based on:

- Introduction to OpenMP, by Ruud van der Pas (www.openmp.org)
- Introduction to OpenMP tutorial from LLNL (www.openmp.org)
- The OpenMP course developed at HLRS Stuttgart
- The OpenMP slides and Videos by Tim Mattson (Intel) (https://www.openmp.org/resources/tutorials-articles/)
- Parallel Programming for Science Engineering by Victor Eijkhout (https://theartofhpc.com/pcse/)

Class Outline

- OpenMP Overview
- OpenMP Execution Model
- OpenMP Memory Model
- Implicit and Explicit Data-Scoping
- Data persistency in OpenMP

OpenMP Basics

OpenMP is an API for parallelism in C/C++, and FORTRAN programs

OpenMP consists of a set of:

- **compiler directives (C/C++ #pragma), library routines, and environment variables.**

Originally intended to parallelize loop-centric algorithms:

→ data parallelism

Since version 3.0: explicit tasks supported:

→ task parallelism

OpenMP provides the support for:

- **work-sharing, synchronization, tasking, and other constructs.**

OpenMP Overview

OpenMP has a very simple premise:

- **programmer** identifies parallelization opportunities
- **compiler** implements the parallel code

C++ serial code

```
for(int i = 0; i < 1000000; i++)
{
    array[i] *= 2;
}
```

Parallelization is identified by **pragmas**

Parallel version (C++ and OpenMP pragma)

```
#pragma omp parallel for
for(int i = 0; i < 1000000; i++)
{
    array[i] *= 2;
}
```

OpenMP Overview

OpenMP has a very simple premise:

- **programmer** identifies parallelization opportunities
- **compiler** implements the parallel code

Programmer identifies parallelization with **pragmas:**

- minimal changes between serial and parallel version
- closer to algorithm and faster development time.

C++ serial code

```
for(int i = 0; i < 1000000; i++)
{
    array[i] *= 2;
}
```

```
//c++ threads
std::vector<std::thread> threads;
int chunksize = 1000000 / NUM_THREADS;
for (int i = 0; i < NUM_THREADS; i++) {
    int min = ...;
    int max = ...;
    threads.emplace_back([&array, min, max]() {
        for (int i = min; i < max; i++) {
            array[i] *= 2;
        }});
}
for (auto& thread : threads) {thread.join();}
```

```
// OpenMP implementation (1 line added)
#pragma omp parallel for
for(int i = 0; i < 1000000; i++)
{
    array[i] *= 2;
}
```

OpenMP Overview

OpenMP has a very simple premise:

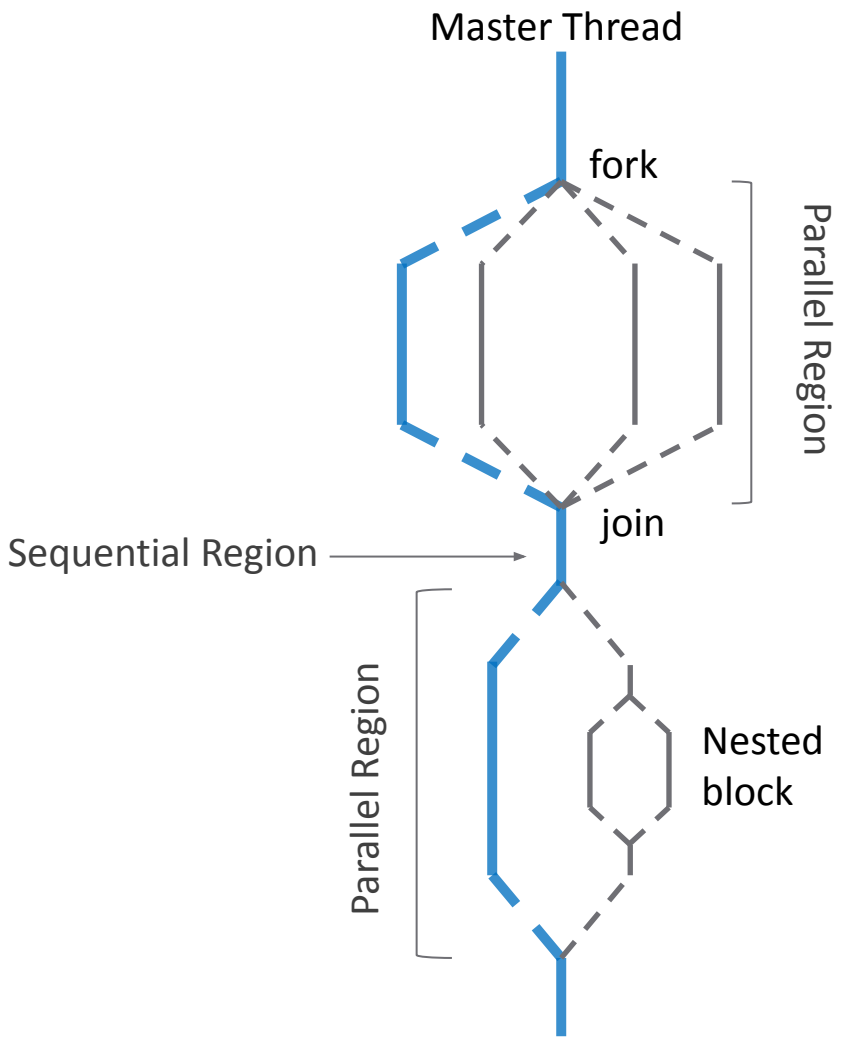
- **programmer** identifies parallelization opportunities
- **compiler** implements the parallel code

Programmer identifies parallelization with **pragmas:**

- minimal changes between serial and parallel version
- **closer to algorithm** and **faster development time**.

Fork-Join execution model

- **parallel region** between **fork** and **join**
- **all threads** execute the same code
- **workload** can be **divided** among threads



Fork and join model

OpenMP Overview

OpenMP has a very simple premise:

- **programmer** identifies parallelization opportunities
- **compiler** implements the parallel code

Programmer identifies parallelization with **pragmas:**

- minimal changes between serial and parallel
- **closer to algorithm** and **faster development time**.

Fork-Join execution model

- **parallel region** between **fork** and **join**
- **all threads** execute the same code
- **workload** can be **divided** among threads

OpenMP allows **incremental parallelization**

- independent **parallel regions**
- **environment variables** change code behavior

C++ serial code

```
int N = 999999;
int sum = 0;

...

for(int i = 0; i < N; i++){
    sum += A[i];
}
```

Correct but not efficient!

```
int N = 999999;
int sum = 0;

...
#pragma omp parallel for
for(int i = 0; i < N; i++){
    #pragma omp critical
     sum += A[i];
}
```

Correct and efficient!

```
int N = 999999;
int sum = 0;

...
#pragma omp parallel for reduction(+:sum)
for(int i = 0; i < N; i++) {
     sum += A[i];
}
```

OpenMP Overview

OpenMP has a very simple premise:

- **programmer** identifies parallelization opportunities
- **compiler** implements the parallel code

Programmer identifies parallelization with **pragmas:**

- minimal changes between serial and parallel

Fork-Join execution model

- **parallel region** between **fork** and **join**
- **all threads** execute the same code
- **workload** can be **divided** among threads

OpenMP allows **incremental parallelization**

- independent **parallel regions**
- **Environment variables** modify behaviour

OpenMP specifies an **interface** and **expected behaviour**

- the **compiler implementation** depends on **OS** and **machine architecture**.
- **OpenMP interface** is independent of **compiler**, **OS** or **architecture**

↓

OpenMP code is **portable** to many different combinations of **Operating Systems** and **Machine Architectures**

OpenMP Basics: Core Syntax

Include file with relevant function prototypes and types:

```
#include <omp.h>
```

C/C++ preprocessing directives (pragmas) take the form:

```
#pragma omp construct [construct] clause [clause] …
{
    Structured-block
}
```

Structured block is:
for C/C++, an executable statement, possibly compound, with a single entry at the top and a single exit at the bottom, or an OpenMP construct. (from OpenMP Specification)

OpenMP: Program Structure (C/C++)

OpenMP header

OpenMP directive

OpenMP routine

```
#include <omp.h>

int main(int argc, char* argv[]) {
   // some code
   #pragma omp construct clause [clause] ... ]
   {
      // more code
      int tid = omp_get_thread_num();
   }
   // rest of the code
   return 0;
}
```

structured-block

Compile and execute (with gcc):

```
> g++ mycode.cpp -o myprogram –fopenmp
> export OMP_NUM_THREADS=4
> ./myprogram
```

OpenMP flag

Environment variable

OpenMP Basics: Run Time Routines

```
omp_set_num_threads(int number_of_threads):
    -Dynamically set the number of threads to use within a parallel region.
int omp_get_thread_num(void):
    -Returns the unique thread_id of the thread that called this routine.
int omp_get_num_threads(void):
    -Returns the current number of threads present in the current team.
int omp_get_max_threads(void):
    -Returns the maximum number of threads that can execute a parallel region.
double omp_get_wtime(void):
    -Returns the wall time in seconds passed since a given point in the past.
```

Note: **Ideally we want to avoid routines, and use clauses and environment vars instead!**

omp parallel

Spawns a team of threads at the fork

- Team = Master (ID = 0) + Workers (ID > 0)

Each member of the team executes the same code found within this parallel region

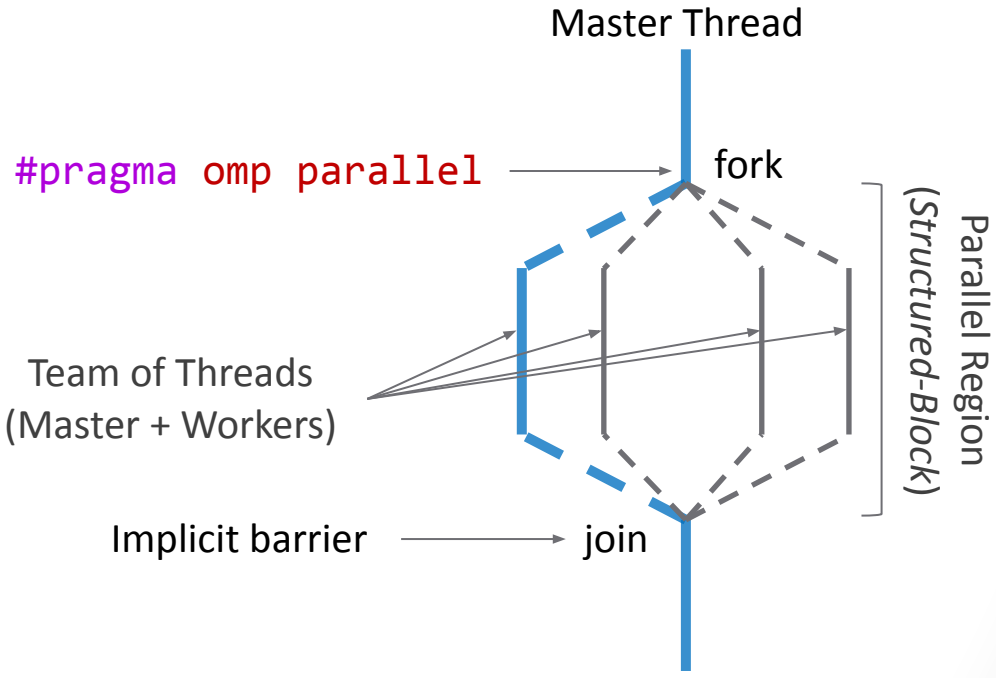
Implicit barrier at the end of the parallel region

- **All threads must reach the barrier before leaving the parallel region.**

Number of requested threads is not guaranteed, but can be requested by the programmer, with:

- **`num_threads`** (clause)
- **`omp_set_num_threads`** (run-time routine)
- **`OMP_NUM_THREADS`** (environment variable)

```
#pragma omp parallel [clauses]
{
   Structured-block
}
```



omp parallel: Hello World

```
int main(int argc, char* argv[])
{
   cout << "Hello World!\n";
   return 0;
}
```

Serial C++ code

```
>./hello_world.cpp
Hello World!
>
```

Terminal

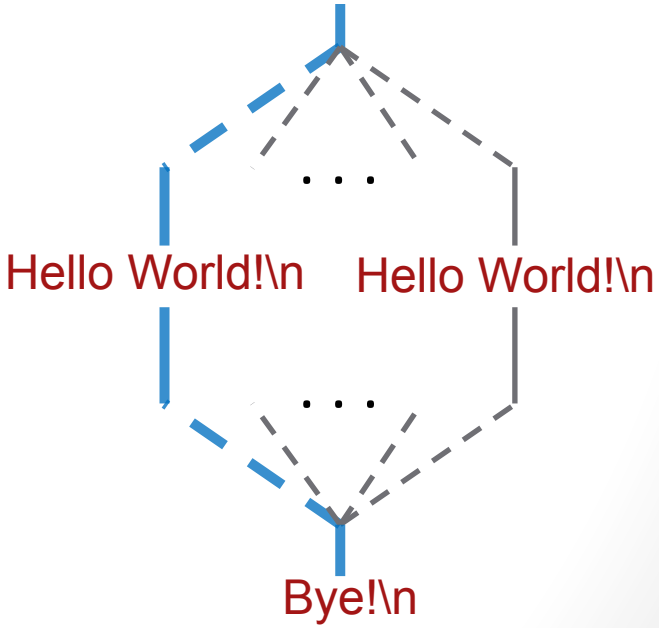
```
#include <omp.h>

int main(int argc, char* argv[])
{
   #pragma omp parallel
   {
     cout << "Hello World!\n";
   }
   cout << "Bye!\n";
   return 0;
}
```

OpenMP code

```
>./hello_world.cpp
Hello World!
Hello World!
...
Hello World!
Bye!
>
```

Terminal



OpenMP: Memory Model

In OpenMP each thread has its own stack

- Memory region private to the thread that owns it

  (**<u>only the owner thread has access to it</u>**)

- Private data is stored/copy in this memory region
- **var1** and **var2** are considered **<u>private variables</u>**

If a thread accesses/modifies a private variable, the thread is accessing/modifying its own private copy of the variable (located in its private stack).

Thus, a modification of var1 by thread 0 does not impact the value of var1 owned by thread 1.

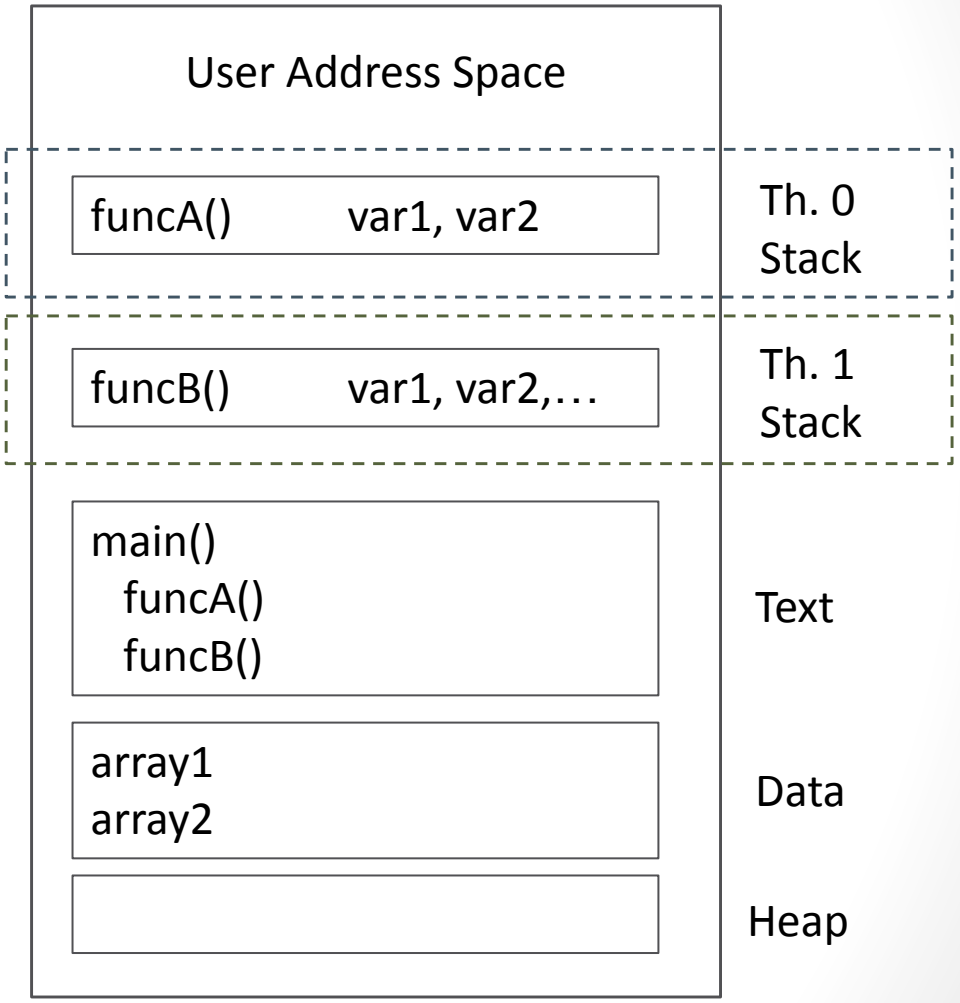


Diagram of a Process

Source: Tim Mattson (Intel) www.openmp.org

OpenMP: Memory Model

Threads share text region, data region and heap:

- Data stored in these regions is accessible to all threads
- Shared variables are allocated in the data region
- **array1** and **array2** are considered **<u>shared variables</u>**

Any thread that accesses/modifies a shared variable references the same memory address located in the process data region.

If a thread modifies the value of a shared value, at some point in time, all threads will observe this change.

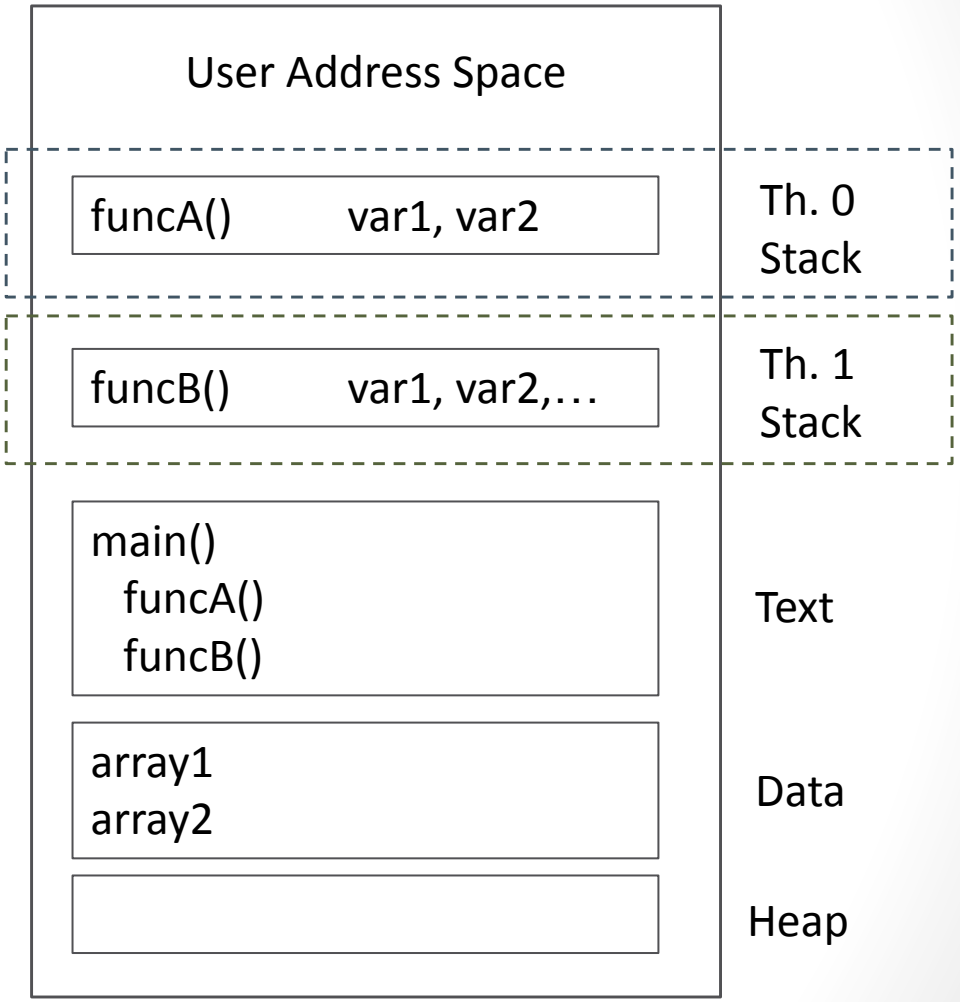


Diagram of a Process

Source: Tim Mattson (Intel) www.openmp.org

OpenMP: Data Scoping

The data scope attributes of a variable serves different purposes:

- **Shared data** - allows establishing communication between threads.
- **Private data** - allows independent execution and helps debugging.

OpenMP allow us to define the data-scope of a variable in two different ways:

- **Implicitly** - the data-scope of the variable depends on its **location (in the code)**,
- **Explicitly** - the data-scope of the variable is explicitly declared inside a **clause.**

default(none): this clause **enforces explicit definition** of the data-scope. **(Good practice)**

default(share): default behavior of OpenMP. **(Easy to commit mistakes)**

The **explicit data-scoping** has priority over **implicit data-scoping.**

Implicit Data Scoping

OpenMP allows the **programmer** to implicitly define the data scope of each variable

- By default, a variable declared outside of the **parallel region** is allocated in a **shared region:**
  - **This variable is shared** - meaning all **threads** can **read** and **write** to this **memory-address**.

```
...
int A = 5;

#pragma omp parallel num_threads(2)
{
    printf("TID:%d - A:%d\n", omp_get_thread_num(), A);

    if(omp_get_thread_num()==0) { A = 3;}
}
printf("After parallel region:%d\n", A);
...
```

"**A**" is considered a **shared variable**, i.e. this **variable** is **allocated** in a **shared memory-address**

```
possible output:
(assume thread 0 prints and modifies A
before thread 1 prints)

TID:0 - A:5

TID:1 - A:3

After parallel region: 3
```

Implicit Data Scoping

OpenMP allows the **programmer** to implicitly define the data scope of each variable

- By default, a variable declared outside the **parallel region** is allocated in a **shared region:**
  - **This variable is shared** - meaning all **threads** can **read** and **write** to this **memory-address**.

- Variables declared inside the **parallel region** are copied to **private stack** of each **thread:**
  - **This variable is private** - only the **owner thread** has **access** to its **memory-address**.

```
...
#pragma omp parallel num_threads(2)
{
    int B = omp_get_thread_num();

    printf("TID:%d - B:%d\n", omp_get_thread_num(), B);
}
...
```

"B" is considered a **private variable**, i.e. this **variable** is **allocated** in the **private memory** of each thread

```
possible output:

TID:0 - B:0

TID:1 - B:1
```

Exercise:

What is printed by each thread:

Thread 0 (master):	Thread 1 (worker):
A0: ?	A0: ?
B0: ?	B0: ?
B1: ?	B1: ?
A1: ?	A1: ?
A2: ?	A2: ?

```
int main(int argc, char* argv[]) {

    int A = 5;

    #pragma omp parallel num_threads(2)
    {
        int B;

        if(omp_get_thread_num()==1)
        {
            printf("A0:%d\n", A);
        }

        printf("B0:%d\n", B);

        B = omp_get_thread_num();

        printf("B1:%d\n", B);

        if(omp_get_thread_num()==1)
        {
            A = 8;
            printf("A1:%d\n", A);
        }
    }
    printf("A2:%d\n", A);
    return 0;
}
```

Exercise:

What is printed by each thread:

Thread 0 (master):	**Thread 1 (worker):**
B0: 0/NaN	A0: 5
B1: 0	B0: 0/NaN
A2: 8	B1: 1
	A1: 8

```
int main(int argc, char* argv[]) {

    int A = 5;

    #pragma omp parallel num_threads(2)
    {
        int B;

        if(omp_get_thread_num()==1)
        {
            printf("A0:%d\n", A);
        }

        printf("B0:%d\n", B);

        B = omp_get_thread_num();

        printf("B1:%d\n", B);

        if(omp_get_thread_num()==1)
        {
            A = 8;
            printf("A1:%d\n", A);
        }
    }
    printf("A2:%d\n", A);
    return 0;
}
```

Explicit Data Scoping

OpenMP allows the **programmer** to **explicitly define** the **data scope** of variables through **clauses**:

- `#pragma omp parallel` **`shared(list)`** - `list` elements are explicitly **shared**.

```
omp_set_num_threads(2);

int A = 1;
#pragma omp parallel default(none) shared(A)
{
    if (omp_get_thread_num() == 0) {
        A = 0;
    }

    if (omp_get_thread_num() == 1) {
        printf("In parallel region: %d\n", A);
    }
}
printf("After parallel region:%d\n", A);
```

```
Possible Outputs:

???
```

Explicit Data Scoping

OpenMP allows the **programmer** to **explicitly define** the **data scope** of variables through **clauses**:

- `#pragma omp parallel` **`shared(list)`** - `list` elements are explicitly **shared**.

```
omp_set_num_threads(2);

int A = 1;
#pragma omp parallel default(none) shared(A)
{
    if (omp_get_thread_num() == 0) {
        A = 0;
    }

    if (omp_get_thread_num() == 1) {
        printf("In parallel region: %d\n", A);
    }
}
printf("After parallel region:%d\n", A);
```

```
Possible Outputs:

(Assuming t0 writes to A before t1 reads it)

In parallel region: 0

After parallel region: 0


(Assuming t0 writes to A after t1 reads it)

In parallel region: 1

After parallel region: 0
```

Explicit Data Scoping

OpenMP allows the **programmer** to **explicitly define** the **data scope** of variables through **clauses**:

- `#pragma omp parallel shared(list)`
- `#pragma omp parallel` **`private(list)`** - `list` elements are explicitly **private**.

```
int A = 1;

#pragma omp parallel default(none) private(A)
{
    int TID = omp_get_thread_num();

    printf("TID:%d - A0:%d\n", TID, A);

    if(TID == 0) //master thread (tid=0)
        A = 5;

    printf("TID:%d - A1:%d\n",TID, A);
}
printf("After parallel region:%d\n", A);
```

```
Possible Output:
(assume prints are interleaved)

TID: 0 - A0: ?

TID: 1 - A0: ?

TID: 0 - A1: ?

TID: 1 - A1: ?

After parallel region: ?
```

Explicit Data Scoping

OpenMP allows the **programmer** to **explicitly define** the **data scope** of variables through **clauses**:

- `#pragma omp parallel shared(list)`
- `#pragma omp parallel` **`private(list)`** - `list` elements are explicitly **private**.

```
int A = 1;

#pragma omp parallel default(none) private(A)
{
    int TID = omp_get_thread_num();

    printf("TID:%d - A0:%d\n", TID, A);

    if(TID == 0) //master thread (tid=0)
        A = 5;

    printf("TID:%d - A1:%d\n",TID, A);
}
printf("After parallel region:%d\n", A);
```

```
Possible Output:
(assume prints are interleaved)

TID: 0 - A0: 0 or <uninitialized>

TID: 1 - A0: 0 or <uninitialized>

TID: 0 - A1: 5

TID: 1 - A1: 0 or <uninitialized>

After parallel region: 1
```

Explicit Data Scoping

OpenMP allows the **programmer** to **explicitly define** the **data scope** of variables through **clauses**:

- `#pragma omp parallel shared(list)`
- `#pragma omp parallel private(list)`
- `#pragma omp parallel` **`firstprivate(list)`**-list elements are **private**, but **initialized**!

```
int A = 1;

#pragma omp parallel default(none) firstprivate(A)
{
    int TID = omp_get_thread_num();

    printf("TID:%d - A0:%d\n", TID, A);

    if(TID == 0) //master thread (tid=0)
        A = 5;

    printf("TID:%d - A1:%d\n",TID, A);
}
printf("After parallel region:%d\n", A);
```

```
Possible Output:
(assume prints are interleaved)

TID: 0 - A0: ?

TID: 1 - A0: ?

TID: 0 - A1: ?

TID: 1 - A1: ?

After parallel region: ?
```

Explicit Data Scoping

OpenMP allows the **programmer** to **explicitly define** the **data scope** of variables through **clauses**:

- `#pragma omp parallel shared(list)`
- `#pragma omp parallel private(list)`
- `#pragma omp parallel` **`firstprivate(list)`**-list elements are **private**, but **initialized**!

```
int A = 1;

#pragma omp parallel default(none) firstprivate(A)
{
    int TID = omp_get_thread_num();

    printf("TID:%d - A0:%d\n", TID, A);

    if(TID == 0) //master thread (tid=0)
        A = 5;

    printf("TID:%d - A1:%d\n",TID, A);
}
printf("After parallel region:%d\n", A);
```

```
Possible Output:
(assume prints are interleaved)

TID: 0 - A0: 1

TID: 1 - A0: 1

TID: 0 - A1: 5

TID: 1 - A1: 1

After parallel region: 1
```

OpenMP Parallel: Data Persistency

Private data becomes inaccessible **outside of the parallel region.**

Issue - we might need access to private variables outside of the parallel region:

```
int num_threads = 0;
#pragma omp parallel private(num_threads)
{
    if(omp_get_thread_num() == 0)
        num_threads = omp_get_num_threads();
}
printf("number of threads: %d\n", num_threads);
```

`num_threads` rolled back to value **before parallel region!**

Solution - ?

OpenMP Parallel: Data Persistency

Private data becomes inaccessible **outside of the parallel region.**

Issue - **we might need access to private variables outside of the parallel region:**

num_threads implicitly **shared**

```
int num_threads = 0;
#pragma omp parallel private(num_threads)
{
    if(omp_get_thread_num() == 0)
        num_threads = omp_get_num_threads();
}
printf("number of threads: %d\n", num_threads);
```

Solution - **store relevant private data in a shared variable before leaving parallel region!**

OpenMP Parallel: Data Persistency

Private data becomes inaccessible **outside of the parallel region.**

Issue **- what if we would like to access data of <u>multiple threads</u> after the parallel region:**

```
#pragma omp parallel
{
    double x = foo(...);
}
```

Solution - ?.

OpenMP Parallel: Data Persistency

Private data becomes inaccessible **outside of the parallel region.**

Issue - **what if we would like to access data of <u>multiple threads</u> after the parallel region:**

```
int *myarray = malloc(sizeof(int) * omp_get_max_threads());
#pragma omp parallel default(none) shared(array){
    myarray[omp_get_thread_num()] = foo(...);
}
//value of foo for thread tid is stored in array[tid]
for(int tid=0; tid < omp_get_max_threads(); tid++)
    printf("array[%d]: %d\n", tid, array[tid]);
```

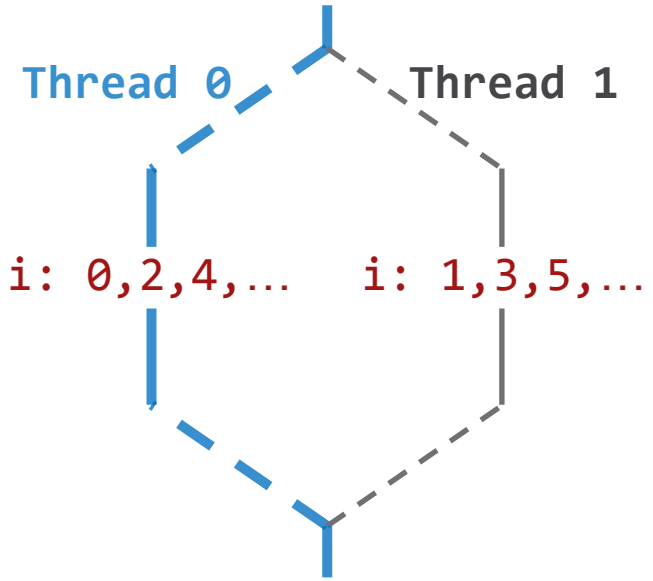
Solution - **Same strategy as before, but now we create a shared variable per thread.**

Simple parallel loop

The `parallel` construct and **`omp_get_thread_num`** routine enable **simple parallel loops**:

```
// example of parallel loop with cyclic distribution
#pragma omp parallel num_threads(2)
{
    int thread_id = omp_get_thread_num();
    int n_threads = omp_get_num_threads();

    for(int i = thread_id; i < iterations; i += n_threads){
        //do some work here
    }
}
```



Note: the code above is not expected in OpenMP. To create a parallel loop use `#omp for` instead!

Exercise:

Q: How can we parallelize the PI program with OpenMP?

- **Hint** - use the **simple parallel loop** and enforce **data persistency.**

```c
//sequential pi program
...
double sum = 0.0;
double step = 1.0 / iterations;

for (i = 0; i < iterations; i++) {
    double x = (i + 0.5) * step;
    sum += 4.0 / (1.0 + x * x);
}

printf("pi: %d\n", sum * step);
```

```c
// example of parallel loop with cyclic distribution
#pragma omp parallel num_threads(2)
{
    int thread_id = omp_get_thread_num();
    int n_threads = omp_get_num_threads();

    for(int i=thread_id; i<iterations; i+=n_threads){
        //do some work here
    }
}
```

Note: the code above is not expected in OpenMP. To create a parallel loop use `#omp for` **instead!**

Exercise:

Q: How can we parallelize the PI program with OpenMP?

- **Hint** - use the **simple parallel loop** and enforce **data persistency.**

```
//sequential pi program
...
int iterations = 100000000;
double sum = 0.0;
double step = 1.0 / iterations;

for (i = 0; i < iterations; i++) {
    double x = (i + 0.5) * step;
    sum += 4.0 / (1.0 + x * x);
}

printf("pi: %d\n", sum * step);
```

1) replace sum with a shared array to store partial values:

```
double partial_sum[omp_get_max_threads()] = {0};
```

2) replace loop with "*simple parallel loop*":

```
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int n_threads = omp_get_num_threads();
    for(int i = tid; i < iterations; i += n_threads) {
        double x = (i + 0.5) * step;
        partial_sum[tid] += 4.0 / (1.0 + x * x);}
}
```

3) accumulate (*reduce*) all partial sums into single variable:

```
for(int i = 0; i < omp_get_max_threads(); i++)
    sum += partial_sum[i];
```

Note: the code above is not expected in OpenMP. To create a parallel loop use `#omp for` **instead!**

OpenMP Terminology

Structured Block: single or a compound statement with a single entry at the top and a single exit at the bottom;

Construct: executable directive applied to the succeeding structured block or to another construct;

Clause: modifier that further specifies the behaviour of a construct;

Parallel Region: structured block of code executed by all threads simultaneously;

Thread Team: Master + Workers;

Master Thread: Initial thread of a program (ID = 0);

Worker Thread: remaining threads (ID >= 1);

Barrier: implicit or explicit synchronization point.